形式语言与编译: 第四次作业

班级: 学号:

习题 7.1

答：

(1) $w = 010010$

该串的 0 块为

$$0, \quad 00, \quad 0.$$

成功路径选择第一个长度为 1 的 0 块作为第一个待比较块，再选择最后一个长度为 1 的 0 块作为第二个待比较块。中间的 00 块被跳过。

对应成功移动为：

$$\begin{aligned}(q_0, 010010, Z_0) &\vdash (q_2, 010010, Z_0)\\ &\vdash (q_3, 10010, 0Z_0)\\ &\vdash (q_4, 0010, 0Z_0)\\ &\vdash (q_4, 010, 0Z_0)\\ &\vdash (q_4, 10, 0Z_0)\\ &\vdash (q_5, 0, 0Z_0)\\ &\vdash (q_6, \varepsilon, Z_0)\\ &\vdash (q_8, \varepsilon, Z_0).\end{aligned}$$

其中：

- $q_2 \to q_3$ 消耗第一个 0 块并把一个 0 压入栈；
- q_4 跳过中间部分；
- $q_5 \to q_6$ 消耗最后一个 0 块并弹出一个 0；
- 最后输入串耗尽，栈回到 Z_0，到达接受状态。

若把图中 q_8 与接受状态 q_7 合并理解，则最后一步可等价地写为

$$(q_6, \varepsilon, Z_0) \vdash (q_7, \varepsilon, Z_0).$$

(2) $w = 100101001$

该串的 0 块为

$$00, \quad 0, \quad 00.$$

成功路径选择第一个长度为 2 的 0 块和最后一个长度为 2 的 0 块。中间长度为 1 的 0 块被跳过。

对应成功移动为：

$$\begin{aligned}(q_0, 100101001, Z_0) &\vdash (q_1, 100101001, Z_0)\\ &\vdash (q_2, 00101001, Z_0)\\ &\vdash (q_3, 0101001, 0Z_0)\\ &\vdash (q_3, 101001, 00Z_0)\\ &\vdash (q_4, 01001, 00Z_0)\\ &\vdash (q_4, 1001, 00Z_0)\\ &\vdash (q_5, 001, 00Z_0)\\ &\vdash (q_6, 01, 0Z_0)\\ &\vdash (q_6, 1, Z_0)\\ &\vdash (q_7, \varepsilon, Z_0).\end{aligned}$$

其中：

- $q_1 \to q_2$ 消耗开头的 1，表示开始检测第一个 0 块；
- $q_2 \to q_3$ 以及 q_3 上的自环共消耗两个 0，并压入两个 0；
- q_4 跳过中间的 0 块；
- $q_5 \to q_6$ 以及 q_6 上的自环共消耗最后两个 0，并弹出两个 0；
- 最后读入 1 后进入接受状态 q_7。

因此，010010 和 100101001 都有成功路径，均被该 PDA 接受。

习题 7.2(2)

答：

要设计 PDA 接受语言

$$L = \{\, w \in \{0,1\}^* \mid \#_0(w) = 2\#_1(w) \,\},$$

其中 $\#_0(w)$ 表示 w 中 0 的个数，$\#_1(w)$ 表示 w 中 1 的个数。

基本思想是维护差值

$$d = \#_0(w) - 2\#_1(w).$$

若 $d > 0$，用栈符号 A 表示多出来的 0；若 $d < 0$，用栈符号 B 表示还缺少的 0；若 $d = 0$，栈中只有栈底符号 Z_0。

读入一个 0 时，差值 d 增加 1；读入一个 1 时，差值 d 减少 2。因此读入 1 时使用一个中间状态 r，把“减 2”拆成两次“减 1”。

(1) 以终结状态方式接受的 PDA

构造

$$P = (Q, \Sigma, \Gamma, \delta, q, Z_0, F),$$

其中

$$Q = \{q, r, f\}, \qquad \Sigma = \{0,1\}, \qquad \Gamma = \{Z_0, A, B\}, \qquad F = \{f\}.$$

状态含义如下：

- q：正常读入状态；
- r：刚读入一个 1，已经完成一次“减 1”，还需要再完成一次“减 1”；
- f：接受状态。

转移函数用表表示如下：

当前状态	读入	弹出	下一状态	压入
q	0	Z_0	q	AZ_0
q	0	A	q	AA
q	0	B	q	ε
q	1	Z_0	r	BZ_0
q	1	A	r	ε
q	1	B	r	BB
r	ε	Z_0	q	BZ_0
r	ε	A	q	ε
r	ε	B	q	BB
q	ε	Z_0	f	Z_0

解释如下：

- 当读入 0 时，若当前有多余的 0，则继续压入 A；若当前有缺少的 0，则弹出一个 B，表示用这个 0 补足一个缺口。

- 当读入 1 时，需要使差值减少 2。所以先在 q 中完成一次减少，再转到 r，由 r 中的 ε 转移完成第二次减少。
- 输入读完后，若栈中只剩 Z_0，说明

$$\#_0(w) - 2\#_1(w) = 0,$$

即 $\#_0(w) = 2\#_1(w)$，于是转到终结状态 f 接受。

(2) 以空栈方式接受的 PDA

若采用空栈接受方式，则可以省略终结状态 f，只需在栈中只剩 Z_0 时将其弹出即可。

构造

$$N = (Q, \Sigma, \Gamma, \delta', q, Z_0),$$

其中

$$Q = \{q, r\}, \qquad \Sigma = \{0, 1\}, \qquad \Gamma = \{Z_0, A, B\}.$$

除最后一条接受转移外，其余转移与上面的 PDA 相同。将

$$q \xrightarrow{\varepsilon, Z_0/Z_0} f$$

改为

$$q \xrightarrow{\varepsilon, Z_0/\varepsilon} q.$$

因此空栈接受方式的转移表为：

当前状态	读入	弹出	下一状态	压入
q	0	Z_0	q	AZ_0
q	0	A	q	AA
q	0	B	q	ε
q	1	Z_0	r	BZ_0
q	1	A	r	ε
q	1	B	r	BB
r	ε	Z_0	q	BZ_0
r	ε	A	q	ε
r	ε	B	q	BB
q	ε	Z_0	q	ε

当输入串读完且栈被清空时，PDA 接受该输入串。

(3) 两种接受方式的比较

终结状态方式是在满足条件时进入接受状态 f，而空栈方式是在满足条件时弹出栈底符号 Z_0。二者的核心识别过程完全相同，区别只在最后一步：

$$\text{终结状态方式：} \quad (q, \varepsilon, Z_0) \vdash (f, \varepsilon, Z_0);$$

$$\text{空栈方式：} \quad (q, \varepsilon, Z_0) \vdash (q, \varepsilon, \varepsilon).$$

因此，这两个 PDA 接受同一个语言

$$\{\, w \in \{0,1\}^* \mid \#_0(w) = 2\#_1(w) \,\}.$$

习题 7.7(1)

答：

例 7.3 中的 PDA 用来识别语言

$$L_{7-3} = \{ww^R \mid w \in \{0,1\}^*\}.$$

根据图 7-5，将该 PDA 写成状态转移表如下：

当前状态	读入	弹出	下一状态	压入
q_0	0	ε	q_0	0
q_0	1	ε	q_0	1
q_0	ε	ε	q_1	ε
q_1	0	0	q_1	ε
q_1	1	1	q_1	ε
q_1	ε	Z_0	q_2	Z_0

其中，q_0 表示读取前半部分并把输入符号压入栈；q_1 表示读取后半部分并与栈顶符号进行匹配；q_2 是接受状态。

结论：该 PDA 不是确定型 PDA。

理由如下：

在状态 q_0 处，若当前剩余输入串以 0 开头，例如当前 ID 为

$$(q_0, 0w, \beta),$$

其中 $\beta \in \Gamma^*$ 是当前栈内容，则该 PDA 至少有两种可能移动。

第一种，使用自环

$$q_0 \xrightarrow{0,\varepsilon/0} q_0,$$

消耗当前输入符号 0，并把 0 压入栈，于是有

$$(q_0, 0w, \beta) \vdash (q_0, w, 0\beta).$$

第二种，使用从 q_0 到 q_1 的 ε 迁移

$$q_0 \xrightarrow{\varepsilon,\varepsilon/\varepsilon} q_1,$$

不消耗输入符号，也不改变栈，于是有

$$(q_0, 0w, \beta) \vdash (q_1, 0w, \beta).$$

因此，从同一个 ID

$$(q_0, 0w, \beta)$$

出发，PDA 可以有两个不同的下一步 ID：

$$(q_0, w, 0\beta)$$

和

$$(q_1, 0w, \beta).$$

这说明它在运行过程中需要非确定地选择：

- 是继续把当前输入符号压栈，认为它还属于前半部分 w；
- 还是通过 ε 迁移进入 q_1，猜测已经到达中点，开始匹配后半部分 w^R。

同理，当当前输入符号为 1 时，也存在类似的非确定选择：

$$(q_0, 1w, \beta) \vdash (q_0, w, 1\beta),$$

同时也可以

$$(q_0, 1w, \beta) \vdash (q_1, 1w, \beta).$$

所以，该 PDA 在 q_0 处必须“猜测”输入串的中点。而确定型 PDA 不允许在同一个 ID 下存在多个可能的下一步移动。

因此，

例 7.3 中的 PDA 不是确定型 PDA。

习题 8.1(2)

答：

题目中的文法为

$$\begin{aligned}
&P \to \check{D}\check{S},\\
&\check{D} \to \varepsilon \mid \check{D}D;,\\
&D \to \text{int } d,\\
&\check{S} \to \varepsilon \mid S;\check{S},\\
&S \to d = E,\\
&E \to d \mid E + i.
\end{aligned}$$

其中，终结符 d 表示标识符，终结符 i 表示整数常量。因此源程序串

$$\text{int } x;\ x = x + 1;$$

在文法符号层面可写为

$$\text{int } d;\ d = d + i;.$$

现验证符号串

$$\check{D}D; d = E; \check{S}$$

是句子

$$\text{int } d;\ d = d + i;$$

的一个句型。

由文法有推导：

$$\begin{aligned} P &\Rightarrow \check{D}\check{S} \\ &\Rightarrow \check{D}D;\check{S} \\ &\Rightarrow \check{D}D;S;\check{S} \\ &\Rightarrow \check{D}D;d = E;\check{S}. \end{aligned}$$

所以

$$\check{D}D;d = E;\check{S}$$

确实是该文法的一个句型。

继续推导可得：

$$\begin{aligned} \check{D}D;d = E;\check{S} &\Rightarrow D;d = E;\check{S} \\ &\Rightarrow \text{int } d;d = E;\check{S} \\ &\Rightarrow \text{int } d;d = E + i;\check{S} \\ &\Rightarrow \text{int } d;d = d + i;\check{S} \\ &\Rightarrow \text{int } d;d = d + i;. \end{aligned}$$

因此它确实是句子

$$\text{int } d;\ d = d + i;$$

的一个句型。

下面判断它是否为规范句型。

规范句型是最右推导中出现的句型。若要从

$$P \Rightarrow \check{D}\check{S}$$

进行最右推导，则当前最右变元是 $\check{S}$，必须优先改写 $\check{S}$。但目标句型

$$\check{D}D;d = E;\check{S}$$

中，最右边的 $\check{S}$ 仍然保留未展开，而其左边的 $\check{D}$ 已经被展开为

$$\check{D}D;.$$

这不可能出现在最右推导过程中。

因此

$$\boxed{\check{D}D;d = E;\check{S} \text{ 不是规范句型.}}$$

下面给出该句型的短语、直接短语和句柄。

对应的推导树结构为：

$$P \Rightarrow \check{D}\check{S} \Rightarrow \check{D}D;\check{S} \Rightarrow \check{D}D;S;\check{S} \Rightarrow \check{D}D;d = E;\check{S}.$$

由此可得短语如下：

相对于的变元	短语
P	$\check{D}D; d = E; \check{S}$
$\check{D}$	$\check{D}D;$
$\check{S}$	$d = E; \check{S}$
S	$d = E$

其中，直接短语为

$$\boxed{\check{D}D;, \quad d = E.}$$

由于句柄只对规范句型而言才定义，而

$$\check{D}D; d = E; \check{S}$$

不是规范句型，所以它没有句柄。即

$$\boxed{\text{该句型无句柄.}}$$

习题 8.2(1)

答：

题目给出的文法为

$$S \to a \mid \Lambda \mid (T), \qquad T \to T, S \mid S.$$

其中 Λ 是题目中的终结符号。要求写出句子

$$(((a, a), \Lambda, (a)), a)$$

的规范归约，并指出每一步的句柄。

规范归约是最右推导的逆过程，即每一步都归约当前规范句型的句柄。

该句子的规范归约如下：

$$
\begin{aligned}
&(((a,a),\Lambda,(a)),a)\\
\Leftarrow\ &(((S,a),\Lambda,(a)),a)\\
\Leftarrow\ &(((T,a),\Lambda,(a)),a)\\
\Leftarrow\ &(((T,S),\Lambda,(a)),a)\\
\Leftarrow\ &(((T),\Lambda,(a)),a)\\
\Leftarrow\ &((S,\Lambda,(a)),a)\\
\Leftarrow\ &((T,\Lambda,(a)),a)\\
\Leftarrow\ &((T,S,(a)),a)\\
\Leftarrow\ &((T,(a)),a)\\
\Leftarrow\ &((T,(S)),a)\\
\Leftarrow\ &((T,(T)),a)\\
\Leftarrow\ &((T,S),a)\\
\Leftarrow\ &((T),a)\\
\Leftarrow\ &(S,a)\\
\Leftarrow\ &(T,a)\\
\Leftarrow\ &(T,S)\\
\Leftarrow\ &(T)\\
\Leftarrow\ &S.
\end{aligned}
$$

下面列出每一步归约时的句柄和所用产生式。

步数	句柄	归约所用产生式
1	a	$S \to a$
2	S	$T \to S$
3	a	$S \to a$
4	T, S	$T \to T, S$
5	(T)	$S \to (T)$
6	S	$T \to S$
7	Λ	$S \to \Lambda$
8	T, S	$T \to T, S$
9	a	$S \to a$
10	S	$T \to S$
11	(T)	$S \to (T)$
12	T, S	$T \to T, S$
13	(T)	$S \to (T)$
14	S	$T \to S$
15	a	$S \to a$
16	T, S	$T \to T, S$
17	(T)	$S \to (T)$

因此，句子

$$(((a, a), \Lambda, (a)), a)$$

可以按照上面的顺序规范归约为初始符号 S。

习题 8.3

答：

题目给出的文法为

$$S \to A, \qquad A \to Ab \mid bBa, \qquad B \to aAc \mid a \mid aAb.$$

先增广该文法，增加新的初始符号 S'：

$$S' \to S.$$

为便于书写，将产生式编号如下：

$$\begin{aligned}
&(0) \quad S' \to S,\\
&(1) \quad S \to A,\\
&(2) \quad A \to Ab,\\
&(3) \quad A \to bBa,\\
&(4) \quad B \to aAc,\\
&(5) \quad B \to a,\\
&(6) \quad B \to aAb.
\end{aligned}$$

(1) 构造 LR(0) 项目集规范族

各 LR(0) 项目集如下：

$$I_0 = \{S' \to \cdot S,\ S \to \cdot A,\ A \to \cdot Ab,\ A \to \cdot bBa\},$$

$$I_1 = \{S' \to S\cdot\},$$

$$I_2 = \{S \to A\cdot,\ A \to A \cdot b\},$$

$$I_3 = \{A \to b \cdot Ba,\ B \to \cdot aAc,\ B \to \cdot a,\ B \to \cdot aAb\},$$

$$I_4 = \{A \to Ab\cdot\},$$

$$I_5 = \{A \to bB \cdot a\},$$

$$I_6 = \{B \to a \cdot Ac,\ B \to a\cdot,\ B \to a \cdot Ab,\ A \to \cdot Ab,\ A \to \cdot bBa\},$$

$$I_7 = \{A \to bBa\cdot\},$$

$$I_8 = \{B \to aA \cdot c,\ B \to aA \cdot b,\ A \to A \cdot b\},$$

$$I_9 = \{A \to Ab\cdot,\ B \to aAb\cdot\},$$

$$I_{10} = \{B \to aAc\cdot\}.$$

用状态转移表表示 itemDFA 如下：

状态	S	A	B	a	b	c
I_0	I_1	I_2			I_3	
I_1						
I_2					I_4	
I_3			I_5	I_6		
I_4						
I_5				I_7		
I_6		I_8			I_3	
I_7						
I_8					I_9	I_{10}
I_9						
I_{10}						

(2) 说明该文法不是 LR(0) 文法

观察项目集 I_2：

$$I_2 = \{S \to A\cdot,\ A \to A \cdot b\}.$$

其中

$$S \to A\cdot$$

是归约项目，而

$$A \to A \cdot b$$

说明在输入符号 b 上还可以移进。在 LR(0) 分析中，归约项目不看后继输入符号，会在所有终结符上归约，因此在状态 I_2 中产生移进–归约冲突。

另外，项目集 I_9 中有两个归约项目：

$$I_9 = \{A \to Ab\cdot,\ B \to aAb\cdot\},$$

在 LR(0) 分析中会产生归约–归约冲突。

所以该文法不是 LR(0) 文法。

该文法不是 LR(0) 文法。

(3) 计算 FOLLOW 集

由产生式

$$S' \to S$$

可知

$$FOLLOW(S) = \{\#\}.$$

由

$$S \to A$$

可知

$$FOLLOW(S) \subseteq FOLLOW(A),$$

所以

$$\# \in FOLLOW(A).$$

由

$$A \to Ab$$

可知

$$b \in FOLLOW(A).$$

由

$$B \to aAc$$

可知

$$c \in FOLLOW(A).$$

由

$$B \to aAb$$

可知

$$b \in FOLLOW(A).$$

因此

$$FOLLOW(A) = \{\#, b, c\}.$$

又由

$$A \to bBa$$

可知

$$a \in FOLLOW(B).$$

所以

$$FOLLOW(B) = \{a\}.$$

综上：

$$\boxed{FOLLOW(S) = \{\#\}, \qquad FOLLOW(A) = \{\#, b, c\}, \qquad FOLLOW(B) = \{a\}.}$$

(4) 构造 SLR(1) 分析表

记 s_i 表示移进到状态 I_i，r_j 表示按第 (j) 条产生式归约。则 SLR(1) 分析表如下：

状态	a	b	c	$\#$	S	A	B
I_0		s_3			I_1	I_2	
I_1				acc			
I_2		s_4		r_1			
I_3	s_6						I_5
I_4		r_2	r_2	r_2			
I_5	s_7						
I_6	r_5	s_3				I_8	
I_7		r_3	r_3	r_3			
I_8		s_9	s_{10}				
I_9	r_6	r_2	r_2	r_2			
I_{10}	r_4						

下面说明该 SLR(1) 分析表没有冲突。

- 在 I_2 中，归约项目 $S \to A\cdot$ 只在

$$FOLLOW(S) = \{\#\}$$

上填入 r_1，而移进项目 $A \to A \cdot b$ 只在 b 上移进，所以没有冲突。

- 在 I_6 中，归约项目 $B \to a\cdot$ 只在

$$FOLLOW(B) = \{a\}$$

上填入 r_5，而移进动作在 b 上发生，因此没有冲突。

- 在 I_9 中有两个归约项目：

$$A \to Ab\cdot, \qquad B \to aAb\cdot.$$

其中 $A \to Ab\cdot$ 只在

$$FOLLOW(A) = \{\#, b, c\}$$

上归约，而 $B \to aAb\cdot$ 只在

$$FOLLOW(B) = \{a\}$$

上归约。两个 FOLLOW 集不相交，因此没有归约–归约冲突。

所以该文法的 SLR(1) 分析表无冲突。

该文法是 SLR(1) 文法，但不是 LR(0) 文法。

习题 9.1(1)

答：

给定文法

$$G_{(e)}: \quad S \to e \mid (S) \mid SS.$$

其中终结符 e 表示原子表达式。要求为 $G_{(e)}$ 设计属性文法 $att(G_{(e)})$，使其能够计算句子的括号嵌套深度。

设综合属性 $S.l$ 表示由变元 S 推导出的子串的最大括号嵌套深度。由于题目规定计算次序默认从左往右，因此在候选式 SS 中，令 $S[1]$ 表示左边第一个 S，$S[2]$ 表示右边第二个 S，$S[0]$ 表示产生式左部的 S。

增广文法并配备属性方程如下：

$$\begin{aligned}
&(1) \quad S' \to S \quad \{\, S'.l = S.l \,\},\\
&(2) \quad S \to SS \quad \{\, S[0].l = \max(S[1].l, S[2].l) \,\},\\
&(3) \quad S \to (S) \quad \{\, S[0].l = S[1].l + 1 \,\},\\
&(4) \quad S \to e \quad \{\, S.l = 0 \,\}.
\end{aligned}$$

解释如下：

- 对于原子表达式 e，其中没有括号，因此嵌套深度为 0；
- 对于 (S)，外面新增了一层括号，因此深度为内部 S 的深度加 1；
- 对于 SS，整体深度等于左右两部分深度的较大者；
- S' 只是增广开始符号，其属性值继承自 S。

因此，上述 $att(G_{(e)})$ 可以计算任意句子的括号嵌套深度。

习题 9.1(2)

答：

考虑句子

$$e(e(e))(e).$$

由习题 9.1(1) 中的属性文法：

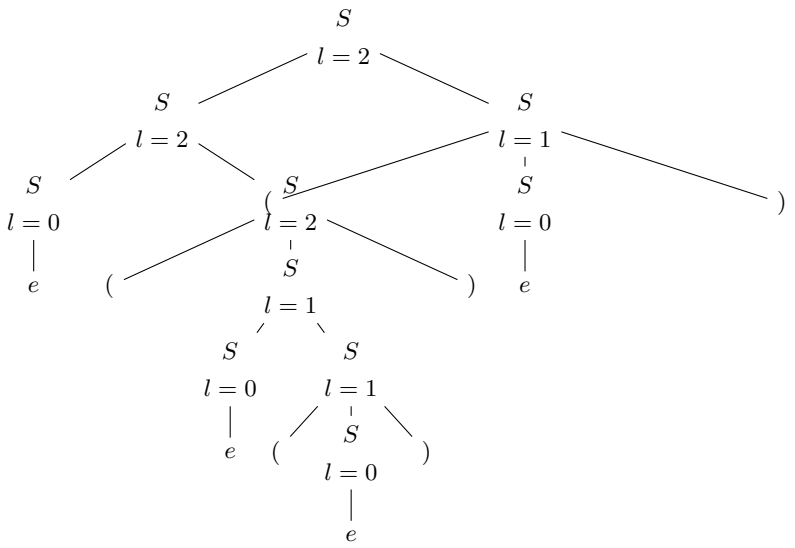
$$\begin{aligned}
&(1)\quad S' \;\to\; S \quad \{\, S'.l = S.l \,\},\\
&(2)\quad S \;\to\; SS \quad \{\, S[0].l = \max(S[1].l, S[2].l) \,\},\\
&(3)\quad S \;\to\; (S) \quad \{\, S[0].l = S[1].l + 1 \,\},\\
&(4)\quad S \;\to\; e \quad \{\, S.l = 0 \,\}.
\end{aligned}$$

(1) 带注释语法树

按从左到右的结合方式，可将句子分解为

$$e(e(e))(e) = \big(e(e(e))\big)(e).$$

其带注释语法树如下：



由语法树可见，最深的一层出现在子串

$$(e(e))$$

内部，其中最内层的 (e) 已经位于第二层括号内，所以整个句子的括号嵌套深度为

$$\boxed{2}.$$

(2) 语法制导的属性求值过程

下面给出类似课件表 9-1 的语法制导属性求值过程。表中 $-$ 表示终结符没有 l 属性；归约时按照对应产生式的属性方程计算 $S.l$。

符号栈	l栈	剩余串	动作
$\#$	$\#$	$e(e(e))(e)\#$	移进
$\#e$	$\#-$	$(e(e))(e)\#$	$r4;\ S.l=0$
$\#S$	$\#0$	$(e(e))(e)\#$	移进
$\#S($	$\#0-$	$e(e))(e)\#$	移进
$\#S(e$	$\#0--$	$(e))(e)\#$	$r4;\ S.l=0$
$\#S(S$	$\#0-0$	$(e))(e)\#$	移进
$\#S(S($	$\#0-0-$	$e))(e)\#$	移进
$\#S(S(e$	$\#0-0--$	$))(e)\#$	$r4;\ S.l=0$
$\#S(S(S$	$\#0-0-0$	$))(e)\#$	移进
$\#S(S(S)$	$\#0-0-0-$	$)(e)\#$	$r3;\ S[0].l=S[1].l+1=1$
$\#S(SS$	$\#0-01$	$)(e)\#$	$r2;\ S[0].l=\max(0,1)=1$
$\#S(S$	$\#0-1$	$)(e)\#$	移进
$\#S(S)$	$\#0-1-$	$(e)\#$	$r3;\ S[0].l=1+1=2$
$\#SS$	$\#02$	$(e)\#$	$r2;\ S[0].l=\max(0,2)=2$
$\#S$	$\#2$	$(e)\#$	移进
$\#S($	$\#2-$	$e)\#$	移进
$\#S(e$	$\#2--$	$)\#$	$r4;\ S.l=0$
$\#S(S$	$\#2-0$	$)\#$	移进
$\#S(S)$	$\#2-0-$	$\#$	$r3;\ S[0].l=0+1=1$
$\#SS$	$\#21$	$\#$	$r2;\ S[0].l=\max(2,1)=2$
$\#S$	$\#2$	$\#$	$r1;\ S'.l=S.l=2$
$\#S'$	$\#2$	$\#$	acc

因此，句子

$$e(e(e))(e)$$

的括号嵌套深度为

$$\boxed{2}.$$

习题 9.3(1)

答：

题目给出的无名函数的声明部分为

```
int x; int a[5]; float b[3,6];
```

按照本章主文法配套的属性文法，声明语句的作用是向当前符号表中添加登记项。假定

$$sizeof(\text{INT}) = 4, \qquad sizeof(\text{FLO}) = 8.$$

同时假定该无名函数的当前符号表初始宽度为 0。对声明从左到右处理。

(1) 处理 `int x;`

x 是简单变量，类型为 INT，占用 4 个存储单位。因此：

$$x.offset = 4.$$

处理后当前符号表宽度为

$$width = 4.$$

(2) 处理 `int a[5];`

a 是一维数组，元素类型为 INT，维长为 5。数组总宽度为

$$5 \times sizeof(\text{INT}) = 5 \times 4 = 20.$$

因此处理后符号表宽度为

$$width = 4 + 20 = 24.$$

于是

$$a.base = 24, \qquad a.dims = 1, \qquad a.dim[0] = 5, \qquad a.etype = \text{INT}.$$

(3) 处理 `float b[3,6];`

b 是二维数组，元素类型为 FLO，维长分别为 3 和 6。数组总宽度为

$$3 \times 6 \times sizeof(\text{FLO}) = 3 \times 6 \times 8 = 144.$$

因此处理后符号表总宽度为

$$width = 24 + 144 = 168.$$

于是

$$b.base = 168, \qquad b.dims = 2, \qquad b.dim[0] = 3, \qquad b.dim[1] = 6, \qquad b.etype = \text{FLO}.$$

所以翻译后的符号表可写为如下属性表表示：

@table : (outer:NIL; width:168; argc:0; arglist:NIL; rtype:NIL; code:NIL;
entry:(name:x; type:INT; offset:4);
entry:(name:a; type:ARRAY; base:24; etype:INT; dims:1; dim[0]:5);
entry:(name:b; type:ARRAY; base:168; etype:FLO; dims:2; dim[0]:3; dim[1]:6)).

其中：

- `offset` 用于简单变量的相对地址；
- `base` 用于数组的存储区基址；
- `dims` 表示数组维数；
- `dim[i]` 表示第 i 维维长；
- `etype` 表示数组元素类型。

习题 9.3(4)

答：

题目给出的当前函数声明部分为：

```
int h(int f(nil); int y;) {
    int g(int c[];);
    { Š };
    Š;
};
```

为了给出符号表表示，设外层当前符号表为 `cur@table`，并假定其初始宽度为 0。若原来已有宽度 W，则下面在 `cur@table` 中新增登记项的偏移量统一加上 W 即可。

(1) 在当前符号表中登记函数 h

函数 `h` 的返回类型为 INT，因此它是一个函数名登记项，类型为 FUNC。按照函数登记项的处理方式，需要为它创建自己的函数符号表 `h@table`，并在当前符号表中记录指向该表的 `mytab` 域。

函数登记项占用两个字，记为

$$sizeof(\text{FUNC}) = 8.$$

所以在当前符号表中有

$$h.offset = 8.$$

于是当前符号表为：

```
cur@table:(
    outer:NIL; width:8; argc:0; arglist:NIL;
    rtype:NIL; code:NIL;
    entry:(name:h; type:FUNC; offset:8; mytab:h@table)
).
```

(2) 构造 h 的函数符号表

函数 `h` 的形参表为：

```
int f(nil); int y;
```

其中，f 是函数原型形参，y 是普通整型形参。

对于形参 f，它表示一个返回类型为 INT 的函数原型形参，将其类型记为 FUNPTT。该类形参占用两个字：

$$sizeof(\text{FUNPTT}) = 8.$$

因此

$$f.offset = 8, \qquad f.rtype = \text{INT}.$$

对于形参 y，它是普通整型形参，占用一个字：

$$sizeof(\text{INT}) = 4.$$

所以

$$y.offset = 12.$$

接着，在 h 的声明部分中出现：

```
int g(int c[];);
```

因此还要在 h@table 中登记函数 g，并为 g 创建函数符号表 g@table。函数登记项占用两个字，所以

$$g.offset = 20.$$

于是 h 的函数符号表为：

```
h@table:(
    outer:cur@table; width:20; argc:2;
    arglist:(f y); rtype:INT; code:[...];
    entry:(name:f; type:FUNPTT; offset:8; rtype:INT);
    entry:(name:y; type:INT; offset:12);
    entry:(name:g; type:FUNC; offset:20; mytab:g@table)
).
```

其中 code:[...] 表示由 $\check{S}$ 翻译得到的代码，题目提示可省略。

(3) 构造 g 的函数符号表

函数 g 的声明为：

```
int g(int c[];);
```

因此，g 的返回类型为 INT，其形参为：

```
int c[];
```

这里 c 是数组原型形参，将其类型记为 ARRPTT。数组原型形参占用一个字：

$$sizeof(\text{ARRPTT}) = 4.$$

所以有

$$c.offset = 4, \qquad c.etype = \text{INT}.$$

于是 g 的函数符号表为：

```
g@table:(
    outer:h@table; width:4; argc:1;
    arglist:(c); rtype:INT; code:NIL;
    entry:(name:c; type:ARRPTT; offset:4; etype:INT)
).
```

(4) 最终层次符号表

综上，该声明翻译后形成三个相关符号表：

$$\texttt{cur@table},\quad \texttt{h@table},\quad \texttt{g@table}.$$

它们之间的层次关系为：

$$\texttt{cur@table} \longrightarrow \texttt{h@table} \longrightarrow \texttt{g@table}.$$

也就是说：

$$\texttt{h@table.outer} = \texttt{cur@table},\quad \texttt{g@table.outer} = \texttt{h@table}.$$

最终属性表表示如下。

```
cur@table:(
    outer:NIL; width:8; argc:0; arglist:NIL;
    rtype:NIL; code:NIL;
    entry:(name:h; type:FUNC; offset:8; mytab:h@table)
).

h@table:(
    outer:cur@table; width:20; argc:2;
    arglist:(f y); rtype:INT; code:[...];
    entry:(name:f; type:FUNPTT; offset:8; rtype:INT);
    entry:(name:y; type:INT; offset:12);
    entry:(name:g; type:FUNC; offset:20; mytab:g@table)
).

g@table:(
    outer:h@table; width:4; argc:1;
    arglist:(c); rtype:INT; code:NIL;
    entry:(name:c; type:ARRPTT; offset:4; etype:INT)
).
```